

(Part 01):السؤال الأول: عشان ده بيقفل الاعتمادية (Coupling) بين أجزاء الكود، وبيزود المرونة (Flexibility). لما تعتمد على إنترفيس، بتقدر تغير الـ Implementation في أي وقت من غير ما الكود الأساسي يتأثر، وده بيسهل الـ Unit Testing والـ Maintenance.

السؤال الثاني: بنستخدم الـ Abstract Class لما يكون عندنا كود فعلي (implementation مشترك) عايزين كل الكلاسات اللي بتورث منه تستخدمه، أو لو محتاجين نعرف fields/variables جواه. إنما الإنترفيس بنستخدمه لو عايزين نحدد "قواعد" بس من غير أي كود مشترك.

السؤال الثالث: بتخلي الكلاس نفسه هو اللي فيه اللوجيك بتاع المقارنة، فبدل ما تكتب كود ترتيب من الصفر كل مرة، بتقدر تستخدم دوال جاهزة زي Array.Sort () مباشرة، وهي هتعرف ترتب الأوبجكتس بناءً على اللوجيك اللي في الـ CompareTo.

السؤال الرابع: الغرض الأساسي هو عمل نسخة جديدة ومستقلة من الأوبجكت (Deep Copy) بحيث لو عندك Reference types، تضمن إن النسختين مش بيشاوروا على نفس المكان في الـ الميموري، والتعديل في واحدة مياثرش على الثانية.

السؤال الخامس: لو الكلاس بيورث من 2 Interfaces فيهم ميثود بنفس الاسم، الـ Explicit implementation بيخليك تقدر تكتب كود مختلف لكل ميثود بناءً على الـ Interface اللي بيستخدمها، وبيخفي الميثود دي من الـ API العادي بتاع الكلاس.

السؤال السادس: المبدأ واحد في حماية الداتا، بس الفرق إن الـ Structs هي Value types مفيهاش وراثة، بينما الـ Classes هي Reference types وبتدعم الوراثة، فبتقدر تستخدم modifiers زي protected لحماية الداتا في الوراثة، وده مش متاح في الـ Struct.

السؤال السابع: الـ Abstraction هو إنك تخفي التعقيد وتظهر بس الحاجات المهمة. علاقته بالـ Encapsulation إنهم بيكملوا بعض؛ الـ Abstraction بيخفي التعقيد في التصميم، والـ Encapsulation بيخفي البيانات نفسها ويحميها من التعديل.

السؤال الثامن: حلت مشكلة كبيرة؛ زمان لو ضفت ميثود لإنترفيس، كان لازم تعدل كل الكلاسات. دلوقتي بالـ Default Implementation تقدر تضيف الميثود وتديها كود افتراضي، وبكده الكلاسات القديمة هتفضل شغالة بدون ما الكود يضرب (Backward Compatibility).

السؤال التاسع: بيدي المبرمج مرونة إنه يكرت Object بأكثر من طريقة بناءً على الداتا اللي متاحة معاه، من غير ما يضطر يملأ كل الخصائص يدويًا.

:(Part 02)

مقالة لينكد إن: LinkedIn

السؤال الأول: معناها إنك تخلي المتغيرات بتاعتك من نوع Interface أو Abstract Class بدل ما تكون من نوع كلاس عادي. ده بيخلي السيستم يقبل أي كلاس جديد يطبق الإنترفيس ده مستقبلاً من غير ما تضطر تغير الكود القديم.

السؤال الثاني: زي ما شرحنا، هو التركيز على التصميم وإخفاء التفاصيل. بنطبقه باستخدام الـ Interfaces والـ Abstract Classes، بحيث اليوزر يتعامل مع دوال زي حساب المساحة من غير ما يهتم المعادلة اللي جوه مكتوبة إزاي.